

Groma

Drone photogrammetry → 3D site model → CCTV coverage planning.

A system that turns drone imagery of an outdoor facility (sports pitches, courts, open grounds) into a measurable 3D model, extracts the structures on it, lets a person approve or reject each one, and then computes what CCTV cameras mounted on those structures can actually see — including how coverage degrades when temporary structures like event tents are erected.

Design documents, in order of depth:

- `docs/design.md` — the compressed engineering design
- `docs/explained.md` — the same design from first principles; read this if any concept here is unfamiliar
- `docs/build-spec.md` — milestones, schemas, API surface, test specifications

Read the design docs before changing anything architectural.

The one idea the whole system rests on

Coverage is **never** computed against the raw photogrammetric mesh. It is computed against a small set of fitted primitives — cylinders for masts, thin boxes for fence runs, boxes for buildings, plus a terrain grid — each of which a person has explicitly accepted, rejected, or reclassified.

That proxy model is the product. The photogrammetry is a cheap way to obtain it.

Consequences that follow, and that code must respect:

- The occlusion model reads `structure WHERE state = 'accepted'` . Nothing else.
 - Every dark cell on a coverage map must be traceable to a named, reviewed object.
 - The kernel runs in milliseconds because it works on tens of primitives, not tens of millions of triangles. Do not “improve” it by ray-tracing the mesh.
-

Non-negotiable conventions

Pan and tilt

```
pan 0° points along -Z, increasing clockwise viewed from above
tilt positive = downward

forward = ( sin(pan) · cos(tilt), -sin(tilt), -cos(pan) · cos(tilt) )
right   = normalise( forward × (0,1,0) )
up      = right × forward
```

Almost every geometry bug in this repository will be a violation of this. If a coverage map appears in the wrong quadrant, check here first. `tests/unit/test_kernel_analytic.py::test_pan_cardinals` (T9) exists solely to catch it.

Coordinate frames

Frame	Units	Where
Storage	Projected CRS metres, <code>site.srid</code>	PostGIS, exports, site plans
Compute	Local ENU metres, rebased to <code>site.origin</code>	Kernel, geometry maths
Display	Same local ENU, Y-up	Renderer

Never put projected coordinates into a float32 or into the renderer. Hong Kong Grid eastings are ~830,000 m, where consecutive float32 values are 6 cm apart. Rebase to the site origin once, on load. See `docs/explained.md` §3.3.

Coverage semantics

Coverage evaluates **vertical targets at 1.6 m above the local terrain surface, with foreshortening on**, against the DORI tiers (25 / 62 / 125 / 250 px/m, IEC EN 62676-4).

$$\text{px/m} = (f_{\text{px}} / d) \cdot \cos \delta \qquad \begin{aligned} f_{\text{px}} &= \text{focal_mm} \times \text{res_y} / \text{sensor_h_mm} \\ \delta &= \text{depression angle} \end{aligned}$$

`eval_height_m` is measured from the DTM, not from the datum. The terrain grid is both an occluder (ray-marched) and the source of per-cell evaluation height.

`CoverageResult.eval_y` records the absolute height actually used.

Flat-ground footprint coverage is a fiction that overstates results by roughly a third and

recommends mountings that cannot recognise anybody. Do not add it as a default or a fallback.

Architecture

- **Dependency direction:** `apps/*` → `packages/*` → `packages/contracts` . Never the reverse. Enforced by `tests/unit/test_import_graph.py` .
- `packages/coverage` **is pure**. No I/O, no database, no framework, no logging of user data. It has to run identically in the worker, the CLI, and the browser via WASM. Keep it that way.
- `packages/contracts` **is the single source of truth** for data shapes. TypeScript types are generated from it; never hand-write a duplicate.

Data rules

- **Measurements always carry uncertainty.** `measurement.uncertainty` is `NOT NULL` . Never format a measurement to more precision than its tolerance — `47.82 m ± 0.03` , not `47.8213 m` .
- **Dimensioning is disabled when** `survey.georef = 'none'` . Scale-free Structure from Motion produces a model that is correct in shape and arbitrary in size, and looks perfect. Return 409, don't warn.
- **Surveys are immutable once complete.** Re-flying a site creates a new survey. Reports reference a specific survey id forever.
- **Rejection is typed** — `noise` / `transient` / `duplicate` — and retained. An untyped rejection loses information you will want next year.

Commands

```
make dev          # docker compose up (postgres+postgis, redis) + uvicorn + vite
make test         # unit suite — must stay under 5 seconds
make test-all    # + integration (testcontainers) + Playwright e2e
make lint         # ruff + mypy + tsc
make seed         # site_alpha fixture into the dev database
make kernel-bench
```

Working on the coverage kernel

Bump `KERNEL_VERSION` in `packages/coverage/kernel.py` for any behavioural change. It is recorded on every `coverage_run` and printed in every report, because these numbers end up in tender documents.

`packages/coverage/reference.py` is a deliberately slow, obviously-correct implementation — plain Python loops, no vectorisation, no broad-phase. When a coverage map looks wrong, run both on a small grid, diff the arrays, find the first differing cell, work backwards. That is far faster than reasoning about vectorised NumPy.

When changing either kernel, change both `packages/coverage` and `apps/web/src/kernel` in the same session, and run the parity test. If they drift, the viewer disagrees with its own reports and nobody notices for weeks.

Never update a golden file to make a test pass without understanding what changed. Explain the movement in the commit message first.

Tests that matter

The dangerous tests in this system are the ones that pass by testing nothing. A coverage assertion like “the array has the right shape and some non-zero values” passes for nearly every bug this codebase can have.

Every test must have an expected value derived independently of the code under test — from arithmetic, from `reference.py`, or from a committed golden file.

The analytic cases, with closed-form answers (see `docs/build-spec.md §6.6`):

	Test
T1	<code>f_px</code> computed two ways agrees to 1e-9
T2	Nadir footprint = $2h \cdot \tan(\text{HFOV}/2) \times 2h \cdot \tan(\text{VFOV}/2)$; peak = <code>f_px</code> / <code>h</code>
T3	Wall shadow ends at $a \cdot (h-e) / (h-w)$ — with <code>h=10</code> , <code>w=3</code> , <code>a=20</code> , <code>e=1.6</code> that is exactly 24.0 m
T4	Grid invariance across 1.0 / 0.5 / 0.25 m within 0.5 pp
T5	Fast kernel vs <code>reference.py</code> within 1e-4
T6	Monotonicity: occluders never help, cameras never hurt

T7	<code>foreshorten=True ≤ foreshorten=False everywhere</code>
T8	Self-occlusion: mounted camera is not blocked by its own mast
T9	Pan cardinals
T10	Terrain occlusion matches the T3 shadow formula for a ridge
T11	On a 5% slope, <code>eval_y == terrain + 1.6</code> within 1 mm
T12	<code>porosity=0.5</code> halves ppm behind an occluder, does not zero it
T13	<code>site_alpha</code> golden fixture within ± 0.3 pp
T14	Import graph direction

Beyond the kernel, two recovery tests are the acceptance criteria for the photogrammetry half:

- **Extraction recovers authored truth.** Run extraction on the synthetic survey and assert it recovers every `site_alpha` primitive — poles within 10 cm, fences within 15 cm, buildings within 25 cm. `docs/build-spec.md` §12.5.
- **Reconstruction recovers rendered poses.** Run ODM or COLMAP on the rendered image set and compare against `poses_truth.json` — $\geq 95\%$ registered, positions within $3\times$ GSD after similarity alignment. §16.2.

T8 exists because every pole-mounted camera in every system like this finds its own pole between itself and half the site, and the symptom (“the west half is blind for no reason”) points nowhere near the cause.

Known traps

1. **Pan/tilt sign errors** produce coverage in the wrong quadrant and look superficially fine.
2. **Silent broadcasting bugs** in the vectorised kernel produce maps that are smooth, plausible and wrong.
3. **Chain-link fencing** is geometrically solid and optically nearly transparent. It is modelled with a `porosity` attenuation factor, not as a boolean occluder.
4. **Seasonal vegetation** — a February survey and a July report describe different sites. Structures flagged `seasonal` are computed both ways.

5. **Height datum confusion** — ellipsoidal vs orthometric vs above-ground-level mixed in one model puts cameras tens of metres underground. Every stored height is labelled.
 6. **Reconstruction that “succeeds” with an empty output.** ODM will report success and hand back a 2 KB LAZ or an all-nodata DSM. `recon/artefacts.py` asserts minimum point counts and a maximum nodata fraction; never skip that check.
 7. **Developing photogrammetry stages against real data.** `packages/recon/fixture.py` and the synthetic ground truth exist so extraction, review and tiling can be developed against geometry whose correct answer is known. Debug there first.
-

Current milestone

M0 — Skeleton. Repo layout, contracts, CI, test harness.

The full milestone list is `docs/build-spec.md` §17: M0 skeleton, M1 coverage kernel, M2 synthetic ground truth, M3 frontend, M4 API and database, M5 jobs and reporting, M6 capture ingest, M7 reconstruction, M8 georeferencing, M9 tiling, M10 structure extraction, M11 review UI, M12 measurement, M13 semantic lift, M14 optimisation, M15 deployment.

All fifteen ship. The ordering is driven by testability, not by scope: the kernel comes first because its bugs are the hardest to see, and the synthetic ground truth (M2) comes second because it is what makes every photogrammetry stage testable without a drone. If schedule pressure appears, reduce `--pc-quality`, not the pipeline.